به‌نام آن‌که جان را فکرت آموخت

دانشگاه صنعتی شریف
دانشکدهٔ مهندسی کامپیوتر
درس امنیت داده و شبکه

تاریخ انتشار: ۲۸ مهر ۱۴۰۵
تاریخ تحویل: ۱۸ آبان ۱۴۰۵
مدرس: سولماز سلیمی

پروژهٔ اول: اعتبارسنجی خودکار وصله‌های امنیتی و آزمون رگرسیون در CI/CD

گروه‌های ۴ نفره نمره: ۲٫۵ از ۲۰

۱. هدف و مسئلهٔ پروژه

اعمال یک وصلهٔ امنیتی (Security Patch) به معنای رفع قطعی آسیب‌پذیری نیست؛ گاهی وصله ناقص است و با ورودی دیگری دور زده می‌شود، و گاهی باعث پسرفت عملکردی (Regression) شده و کارکرد عادی برنامه را خراب می‌کند. هدف اصلی این پروژه، ساخت ابزاری خودکار در پایپ‌لاین CI/CD است که صرفاً یک اسکنر ایستا یا جمع‌آوری‌کنندهٔ گزارش نباشد، بلکه با اجرای واقعی برنامه دو پرسش کلیدی را پاسخ دهد:

۱. آیا خطای امنیتی قبلی، پس از بازتولید در نسخهٔ پایه، در نسخهٔ وصله‌شده و در محدودهٔ آزمون‌های تعیین‌شده همچنان رخ می‌دهد؟

۲. آیا وصله، عملکرد درست و مورد انتظار برنامه را روی ورودی‌های معتبر حفظ کرده است؟

ابزار باید بتواند آسیب‌پذیری‌ها و خطاهای رخ‌داده را به تغییرات کد در وصله (Diff) پیوند دهد و شواهد قابل تکرار تولید کند. مشاهده نشدن خطا در آزمون‌ها، اثبات نبود همهٔ آسیب‌پذیری‌ها نیست.

۲. دامنهٔ پشتیبانی و الزامات پایه

برای یکسان‌سازی ارزیابی و جلوگیری از انتخاب اهداف صرفاً اسکریپتی یا تحلیل‌های سطحی، رعایت موارد زیر برای تمام گروه‌ها الزامی است:

- **هدف مورد بررسی (Target):** برنامه‌های کامپایل‌شدنی به زبان C یا C++ در قالب اجرایی ELF روی سیستم‌عامل لینوکس (x86_64) که ورودی را از فایل، آرگومان خط فرمان یا ورودی استاندارد (`stdin`) دریافت می‌کنند. کد منبع هر دو نسخه باید در دسترس و قابل ساخت باشد. استفاده از اسکریپت‌های پایتون، وب‌اپلیکیشن‌ها یا کدهای تفسیرشونده به عنوان برنامهٔ هدف مجاز نیست. این محدودیت به زبان پیاده‌سازی خود ابزار مربوط نمی‌شود.
- **الزام به اجرای پویا:** اجرای واقعی برنامهٔ هدف و آزمون پویا برای همهٔ گروه‌ها اجباری است. اکتفا به ابزارهای تحلیل ایستا (SAST) یا بررسی متنی کد پذیرفته نمی‌شود.
- **قابلیت تعمیم:** ابزار نباید به نام توابع یا ساختار یک برنامهٔ خاص وابسته باشد. افزودن یک برنامهٔ جدید باید تنها با تغییر فایل پیکربندی، شامل دستور ساخت، مسیر باینری، نحوهٔ ارسال ورودی و مجموعهٔ آزمون‌ها، ممکن باشد.

معیار صحت عملکرد باید پیش از ارزیابی، بر اساس مستندات برنامه یا آزمون‌های مرجع مستقل تعریف و به تأیید مدرس برسد. این معیار شامل رفتار ورودی‌های معتبر و نامعتبر، خروجی و کد خروج مورد انتظار است؛ خروجی نسخهٔ وصله‌شده یا نظر مدل زبانی، به‌تنهایی مرجع صحت نیست.

۳. قابلیت‌های مورد انتظار ابزار

۱.۳. اعتبارسنجی وصله و پیوند دادن یافته به تغییرات (هستهٔ اصلی)

ابزار باید نسخهٔ قبل از وصله، نسخهٔ بعد از وصله و تفاوت کد (Git Diff) را دریافت و مقایسه کند:

- **پیوند دادن یافته به وصله (Patch-to-Finding Correlation):** این بخش جزء کار اصلی پروژه است. ابزار باید نشان دهد خطای برطرف‌شده در آزمون‌ها، باقی‌مانده یا تازه مشاهده‌شده چه ارتباطی با تغییرات وصله دارد. تطبیق پشتهٔ فراخوان (Stack Trace) یا مسیر اجرا با فایل، تابع و بخش تغییرکرده و مقایسهٔ اجرای دو نسخه، شواهد این ارتباط هستند. محل خطا لزوماً روی خط تغییرکرده نیست؛ برای مثال، تغییر یک بررسی طول در تابع فراخواننده می‌تواند بر خطای تابع دیگری اثر بگذارد. صرف هم‌جواری ظاهری یک هشدار ایستا با محل تغییر کافی نیست و در صورت ناکافی بودن شواهد، ارتباط باید نامشخص گزارش شود.
- **تفکیک وضعیت یافته‌ها:** گزارش باید یافته‌ها را به چهار دستهٔ برطرف‌شده در محدودهٔ آزمون‌ها (Fixed)، وصلهٔ ناقص یا باقی‌مانده (Incomplete Fix)، خطای تازه مشاهده‌شده (New Issue) و پسرفت عملکردی (Functional Regression) تفکیک کند. وضعیت نامشخص (Inconclusive) نیز برای شواهد ناکافی لازم است. تازه مشاهده شدن یک خطا به‌تنهایی ثابت نمی‌کند که وصله علت آن بوده است.

هر ورودی خطاساز باید با تنظیمات متناظر روی هر دو نسخه اجرا شود. تشخیص پسرفت عملکردی نیز باید نشان دهد که نسخهٔ پایه رفتار مورد انتظارِ مرجع مستقل را داشته و نسخهٔ وصله‌شده آن را نقض کرده است.

۲.۳. آزمون پویا و گزارش دقیق خطا

ابزار باید نسخهٔ وصله‌شده را هم در برابر آزمون‌های عملکردی، شامل ورودی‌های معتبر، و هم در برابر آزمون‌های مرزی و ورودی‌های نامعتبر بیازماید. در صورت بروز خطا یا توقف غیرمنتظره (Crash)، گزارش باید موارد زیر را به‌صورت مجزا ثبت کند:

- **نوع خطا (Error Type):** مانند سرریز بافر پشته یا هیپ، استفاده پس از آزادسازی (UAF)، دسترسی از طریق اشاره‌گر تهی یا تقسیم بر صفر. این دسته‌بندی با تحلیل علت ریشه‌ای خطا یکسان نیست.
- **روش تشخیص (Detection Mechanism):** مشخص شود خطا با سیگنال سیستم‌عامل، محافظ‌های باینری مانند بررسی قناری و فراخوانی `__stack_chk_fail`، یا ابزارهای تشخیص خطا (Sanitizers) مانند ASan/UBSan مشاهده شده است.
- **نحوهٔ خاتمهٔ برنامه (Termination):** کد خروج (Exit Code)، سیگنال دریافتی مانند `SIGSEGV` یا `SIGABRT`، خروجی استاندارد و خروجی خطا ثبت شوند. عبور از محدودیت زمان یا منابع جداگانه گزارش شود؛ هر توقف برنامه الزاماً آسیب‌پذیری قابل بهره‌برداری نیست.
- **تفکیک بیلد عادی و بیلد ابزارگذاری‌شده:** نتیجهٔ اجرای نسخهٔ عادی باینری (Release Build) نباید با نسخهٔ ابزارگذاری‌شده (Instrumented/Sanitized Build) مخلوط شود. ابزار باید هر دو حالت را جداگانه اجرا و گزارش کند تا مشخص شود خطا در باینری نهایی چگونه بروز می‌کند و در بیلد ابزارگذاری‌شده چه جزئیاتی دارد. مقایسهٔ قبل و بعد باید در بیلدهای متناظر انجام شود.

گروه باید مجموعهٔ آزمون‌های معتبر، مرزی و نامعتبر را طراحی و مستند کند. این مجموعه در هر دو نسخه و در بیلدهای متناظر، با تنظیمات و محدودیت‌های یکسان اجرا شود.

۳.۳. تکرارپذیری، کوچک‌سازی خودکار ورودی و آزمون رگرسیون

هر خطای تأییدشده باید قابل بازتولید باشد:

- **بستهٔ بازتولید (Reproducer Bundle):** به ازای هر خطا، ابزار باید فایل ورودی ایجادکنندهٔ خطا، شناسهٔ دقیق نسخهٔ برنامه (Commit Hash)، هش باینری، تنظیمات ساخت، متغیرهای محیطی مؤثر و دستور کامل اجرا را ذخیره کند تا اجرای مجدد آن همان خطا را تکرار کند. نتیجهٔ حداقل سه بازپخش ثبت و موارد ناپایدار مشخص شوند.
- **کوچک‌سازی خودکار ورودی (Testcase Minimization):** ابزار باید ورودی ایجادکنندهٔ خطا را به‌صورت خودکار، مثلاً با حذف تدریجی بایت‌ها یا روشی مشابه Delta Debugging، کوچک‌تر کند، به‌گونه‌ای که ورودی کوچک‌شده همچنان همان نوع خطا را در محل مرتبط ایجاد کند. هر دو ورودی اولیه و کوچک‌شده باید ذخیره شوند. هر کد خروج غیرصفر، معیار حفظ همان خطا نیست.
- **تولید خودکار آزمون رگرسیون:** ورودی کوچک‌شده باید خودکار به مجموعهٔ آزمون‌های رگرسیون پروژه اضافه شود تا در اجراهای بعدی پایپ‌لاین بررسی گردد. آزمون باید خطای مورد انتظار را روی نسخهٔ پایه آشکار کند و نبود همان خطا و رفتار مورد انتظار را روی نسخهٔ وصله‌شده بررسی کند.

۴.۳. بررسی پایهٔ سخت‌سازی باینری (Binary Hardening)

ابزار باید وضعیت محافظت‌های فایل ELF شامل NX، PIE، RELRO و Stack Canary را همراه با شواهد آن‌ها در نسخهٔ عادی بررسی کند و در صورت تغییر ناخواسته در نسخهٔ جدید، مثلاً غیرفعال شدن یک محافظ در اثر تغییر اسکریپت ساخت، هشدار دهد. سطح RELRO به‌صورت بدون محافظت، جزئی یا کامل گزارش شود. وجود PIE فعال بودن ASLR در محیط اجرا را ثابت نمی‌کند و شواهد قناری در باینری نیز به معنای حفاظت همهٔ توابع نیست.

۵.۳. اجرای خودکار در CI/CD و کدهای خروج استاندارد

ابزار باید از خط فرمان قابل اجرا باشد، یک گزارش خلاصهٔ خوانا برای توسعه‌دهنده و یک خروجی ساختاریافتهٔ JSON تولید کند. کد خروج (Exit Code) ابزار باید وضعیت اجرا را به‌طور دقیق و بدون ابهام مشخص کند تا پایپ‌لاین بتواند بر اساس آن تصمیم بگیرد:

- **کد 0 (`PASS`):** خطای هدف روی نسخهٔ پایه بازتولید شده، در نسخهٔ وصله‌شده و در محدودهٔ آزمون‌ها مشاهده نشده، خطای امنیتی تأییدشدهٔ دیگری در آن ثبت نشده و تمام آزمون‌های عملکردی و بررسی‌های الزامی موفق و کامل بوده‌اند.
- **کد 10:** خطای قبلی همچنان رخ می‌دهد (`VULN_REMAINS`) یا یک خطای حافظه یا امنیتی دیگر با شواهد معتبر تأیید شده است (`NEW_ISSUE`). یک توقف طبقه‌بندی‌نشده به‌تنهایی برای تأیید آسیب‌پذیری کافی نیست.
- **کد 20 (`FUNC_REGRESSION`):** دست‌کم یکی از آزمون‌های عملکردی معتبر که در نسخهٔ پایه موفق بوده، در نسخهٔ وصله‌شده شکست خورده است. این وضعیت مستقل از رفع خطای امنیتی ثبت می‌شود.
- **کد 30 (`BUILD_OR_CONFIG_ERROR`):** برنامهٔ هدف کامپایل نمی‌شود، فایل پیکربندی ناقص است یا محیط اجرا و خود ابزار خطا دارند. این موارد نباید با پیدا شدن خطای امنیتی یا تأیید وصله یکسان حساب شوند.
- **کد 40 (`INCONCLUSIVE`):** بررسی الزامی کامل نشده یا شواهد برای داوری کافی نیست؛ مانند بازتولید نشدن خطای پایه، یافتهٔ ناپایدار یا توقف طبقه‌بندی‌نشده.

در صورت هم‌زمانی چند وضعیت، اولویت کد خروج به ترتیب 30، 10، 20، 40 و 0 است. تمام یافته‌ها و وضعیت‌های مستقل امنیت، عملکرد و تکمیل تحلیل، همراه با شواهد، نسخه‌ها و تنظیمات اجرا در JSON باقی بمانند.

۴. سناریوهای آزمون نهایی و ارزیابی

ابزار شما باید روی دو برنامهٔ مستقل به زبان C/C++ آزمایش و تحویل داده شود که حداقل یکی از آن‌ها یک پروژه یا کتابخانهٔ واقعی و غیراسباب‌بازی باشد. انتخاب هدف‌ها و آماده‌سازی نسخه‌ها بر عهدهٔ گروه است و منشأ کد، وصله‌ها، آزمون‌های مرجع و نتایج مورد انتظار باید در مرحلهٔ طراحی به تأیید مدرس برسند. استفاده از آسیب‌پذیری و وصلهٔ عمومیِ شناخته‌شده مجاز است و کشف آسیب‌پذیری تازه الزامی نیست.

برای هر یک از این دو برنامه، باید چهار نسخهٔ زیر آماده و در مخزن پروژه موجود باشد:

۱. **نسخهٔ آسیب‌پذیر پایه (Vulnerable Base):** دارای حداقل یک خطای حافظهٔ قابل بازتولید است.

۲. **وصلهٔ درست (Correct Patch):** خطا در محدودهٔ ارزیابی تعیین‌شده مشاهده نمی‌شود و آزمون‌های عملکردی نیز موفق هستند. خروجی مورد انتظار در اجرای کامل: کد 0.

۳. **وصلهٔ ناقص (Incomplete Patch):** فقط ورودی اولیهٔ خطا را مسدود می‌کند، اما آسیب‌پذیری با یک ورودی متفاوت که گروه طراحی و در مجموعهٔ آزمون‌ها ثبت می‌کند، همچنان رخ می‌دهد. خروجی مورد انتظار: کد 10.

۴. **وصلهٔ مخرب عملکرد (Regression Patch):** جلوی بروز خطای هدف را در آزمون‌ها می‌گیرد، اما رفتار عادی برنامه را روی بخشی از ورودی‌های معتبر خراب می‌کند. در نبود خطای امنیتی تأییدشدهٔ دیگر، خروجی مورد انتظار: کد 20.

آزمون روز دفاع: علاوه بر موارد بالا، در جلسهٔ دفاع چند وصلهٔ ندیده (Unseen Patches) روی برنامه‌های هدف یا یک برنامهٔ هم‌خانواده، در همین دامنه و با رابط ورودی توافق‌شده، به ابزار شما داده می‌شود تا به‌صورت زنده ارزیابی شوند. پاسخ ابزار نباید بر اساس نام فایل یا شناسهٔ از پیش شناخته‌شدهٔ وصله تعیین شود.

۵. ضوابط اجرایی و تحویل

- **گروه‌بندی:** پروژه در گروه‌های ۴ نفره انجام می‌شود.
- **استفاده از مدل‌های زبانی (LLM):** استفاده از مدل‌های زبانی برای کمک در کدنویسی ابزار یا تحلیل اولیه مجاز است؛ اما منطق اعتبارسنجی وصله در پایپ‌لاین باید متکی بر اجرای واقعی و شواهد عینی باشد و روی شواهد یکسان، نتیجهٔ یکسان تولید کند. تمام اعضای گروه باید بر معماری ابزار و بخش کد مربوط به سهم خود مسلط باشند.
- **دفاع فردی و جداگانه:** در جلسهٔ تحویل، از هر عضو گروه به‌صورت جداگانه پرسش فنی و درخواست تغییر محدود در کد یا پیکربندی می‌شود. سهم دفاع فردی در بخش بارم‌بندی مشخص شده است.
- **محیط اجرا:** تمامی آزمون‌ها در محیط آزمایش ایزوله، مانند کانتینر با محدودیت منابع، و روی برنامه‌های مجاز اجرا شوند.

تحویل شامل کد ابزار، پیکربندی و نسخه‌های دقیق اهداف، آزمون‌ها، بسته‌های بازتولید، گزارش‌های تولیدشده، فایل پایپ‌لاین و راهنمای اجرای مستقل است. طرح و انتخاب هدف‌ها در مرحلهٔ طراحی تأیید می‌شوند و تحویل نهایی در تاریخ درج‌شده در سربرگ انجام می‌شود.

۶. بارم‌بندی و نحوهٔ محاسبهٔ نمره

ارزیابی پروژه از دو بخش تشکیل می‌شود: امتیاز فنی محصول گروه و ارزیابی فردی هر عضو. امتیاز فنی گروه از ۱۰۰ محاسبه می‌شود. نمرهٔ هر فرد با اعمال ضریب دفاع فردی بر امتیاز فنی گروه به دست می‌آید؛ بنابراین نمرهٔ اعضای یک گروه لزوماً یکسان نیست.

بخش ارزیابی فنی گروه	امتیاز
اعتبارسنجی وصله، مقایسهٔ نسخه‌ها و پیوند دادن دقیق خطا به تغییرات وصله (Diff)	۲۵
آزمون پویا، ورودی‌های مرزی، تفکیک بیلد عادی و ابزارگذاری‌شده و دسته‌بندی دقیق خطا	۲۵
تکرارپذیری، کوچک‌سازی خودکار ورودی و تولید آزمون رگرسیون	۱۵
پیاده‌سازی و تشخیص درست سناریوهای وصله روی دو برنامهٔ مستقل و عملکرد روی وصله‌های ندیده	۱۵
اجرای خودکار در CI/CD، خروجی استاندارد JSON و تفکیک کدهای خروج	۱۰
بررسی سخت‌سازی باینری و تشخیص تغییر وضعیت محافظت‌ها	۵
مستندات، طراحی پیمانه‌ای و امکان اجرای مستقل	۵
جمع امتیاز فنی گروه	**۱۰۰**

۱.۶. مبنای ارزیابی فنی

امتیازهای جدول، سقف امتیاز هر بخش‌اند. امتیاز هر قابلیت بر اساس صحت پیاده‌سازی، رعایت الزامات همان بخش، اجرای مستقل و شواهد قابل بازتولید تعیین می‌شود. صرف وجود کد، تولید گزارش یا موفقیت روی چند ورودی انتخابی برای دریافت امتیاز کامل کافی نیست. قابلیت ناقص تنها بابت قسمت‌های پیاده‌سازی‌شده و قابل تأیید امتیاز می‌گیرد. قابلیت پیاده‌سازی‌نشده، پاسخ از پیش تعیین‌شده به جای تحلیل واقعی، یا ادعای فاقد شواهد قابل بررسی، امتیازی ندارد.

ارزیابی روی نسخهٔ نهایی تحویل‌شده و در محیط و دامنهٔ تعیین‌شده در صورت پروژه انجام می‌شود. آزمون‌های ندیده نیز در همین دامنه خواهند بود. نمرهٔ کامل مستلزم تحقق تمام الزامات، عملکرد صحیح روی آزمون‌های ارزیابی و امکان بازتولید مستقل نتایج است. ادعای کامل بودن پروژه یا نمایش اجرای موفق، جایگزین این ارزیابی نیست.

۲.۶. احراز مشارکت و دفاع فردی

هر گروه باید سهم واقعی اعضا را در طراحی، پیاده‌سازی، آزمون، تحلیل نتایج و مستندسازی مشخص کند. مشارکت هر فرد با بررسی خروجی منتسب به او، شواهد فرایند توسعه و پاسخ‌های دفاع احراز می‌شود. تعداد خطوط کد یا تعداد ثبت تغییرات در مخزن، به‌تنهایی ملاک مشارکت نیست.

دفاع هر عضو به‌صورت مستقل و بدون کمک سایر اعضا یا ابزارهای مولد پاسخ انجام می‌شود. امتیاز دفاع از ۱۰ و بر اساس سه محور زیر تعیین می‌شود:

- **۳ امتیاز:** توضیح معماری کلی، تصمیم‌های طراحی و منطق کد بخش منتسب به فرد.
- **۳ امتیاز:** تحلیل یک یافته یا نتیجهٔ آزمون و توضیح ارتباط شواهد با تصمیم ابزار.
- **۴ امتیاز:** اعمال یک تغییر محدود در کد یا پیکربندی و بررسی نتیجهٔ آن با اجرای آزمون، در زمان اعلام‌شده برای دفاع. تغییر خواسته‌شده در دامنهٔ پروژه و متناسب با بخش مشارکت فرد خواهد بود.

شرط دریافت نمرهٔ پروژه، احراز مشارکت مؤثر، کسب حداقل ۵ از ۱۰ در دفاع و دریافت حداقل یک امتیاز در هر یک از سه محور فوق است. در صورت احراز نشدن مشارکت مؤثر یا برآورده نشدن حداقل‌های دفاع، نمرهٔ پروژه برای همان فرد صفر خواهد بود؛ موفقیت محصول گروه یا دفاع سایر اعضا جایگزین این شروط نمی‌شود.

استفاده از مدل‌های زبانی و ابزارهای کمکی در انجام پروژه، مسئولیت دانشجو در فهم، بررسی و دفاع از خروجی را تغییر نمی‌دهد. دانشجو باید بتواند منطق بخش منتسب به خود، فرض‌های آن و شواهد صحت نتایج را توضیح دهد و تغییر محدود خواسته‌شده را انجام دهد.

۳.۶. محاسبهٔ نمرهٔ نهایی هر فرد

برای فردی که شرایط فوق را احراز کرده است، ضریب فردی برابر با امتیاز دفاع او تقسیم بر ۱۰ است. نمرهٔ نهایی از رابطهٔ زیر به دست می‌آید:

$$\text{نمرهٔ فرد} = ۲٫۵ \times \frac{\text{امتیاز فنی گروه}}{۱۰۰} \times \frac{\text{امتیاز دفاع فرد}}{۱۰}$$

برای مثال، با امتیاز فنی گروه ۱۰۰ و دفاع فردی ۶ از ۱۰، نمرهٔ آن فرد ۱٫۵ از ۲٫۵ است. دریافت نمرهٔ کامل پروژه به امتیاز فنی کامل، مشارکت مؤثر و امتیاز کامل دفاع فردی نیاز دارد.